

Short-Horizon Electricity Forecasting for a Steel Plant

ML Pipeline Design & MLOps Analysis

Assignment	Individual Assignment 2
Author	Do Quang Hiep
Student ID	25MS13293
Document date	26 September 2026
Evidence status	Measured historical experiments / Docker project

Executive summary

This report implements and evaluates the steel-plant forecasting system proposed in Assignment 1. The reproducible Docker project uses 35,040 public UCI observations, validates the source, builds past-only features and compares ten predeclared configurations on three chronological development folds. Configuration E06 is selected using pooled development MAE, while E01 is the strongest simple baseline. After October-only alert calibration, the frozen model achieves a locked-test MAE of 4.703 kWh against 7.515 kWh for the baseline, a 37.4% relative reduction over 5,856 intervals. The forecast gate passes; the advisory gate does not pass. MLflow preserves experiment and model lineage; Airflow executes explicit stages; an HTTP replay API demonstrates the saved-model contract. Timestamp sensitivity, subgroup errors, uncertainty and implementation limitations are reported. Model registration is kept separate from production approval. These are measured historical forecasting results, not demonstrated factory energy savings or a live production deployment.

Reproducible source, licensed data, recorded results and assessor run instructions accompany this report.

CONTEXT AND CONTINUITY

From system design to executable evidence

Assignment 1 reference and operational objective

Assignment 1 proposed an advisory system for a small steel plant: predict future 15-minute electricity use, flag high-load periods and let operators review optional auxiliary activity. The stakeholders remain the energy manager, shift operator, finance/plant management and IT/data team. The business motivation is peak-demand reduction without disrupting production. Assignment 2 turns that design into a reproducible historical benchmark and analyses the resulting evidence.

A precise forecast contract

Let t be the end of the latest completed meter interval. Assume its reading arrives by $t+30$ seconds; issue the forecast by $t+60$ seconds for $[t+15 \text{ minutes}, t+30 \text{ minutes})$. This gives 14 minutes before the target starts. For a target row j , the most recent usable measurement is $j-2$. Row $j-1$ is still incomplete and is never a predictor. This is a 30-minute horizon to the target end, not a contemporaneous energy estimate.

Dimension	Implemented benchmark	Production boundary
Data	Bundled UCI CSV, immutable source hash and two timestamp interpretations.	Meter semantics, actual arrival delays and plant-local ingestion still require confirmation.
Models	Three baselines, Ridge, two random forests and four boosting/feature variants.	One chosen regressor; no interacting multi-model control system.
Orchestration	Manual research DAG and daily evidence-verification DAG.	Weekly live retraining and 15-minute live inference remain proposed schedules.
Deployment	Versioned candidate, local bundle and Docker HTTP replay API.	No automatic machine actuation, champion promotion or claim of plant readiness.

Refinements justified by implementation

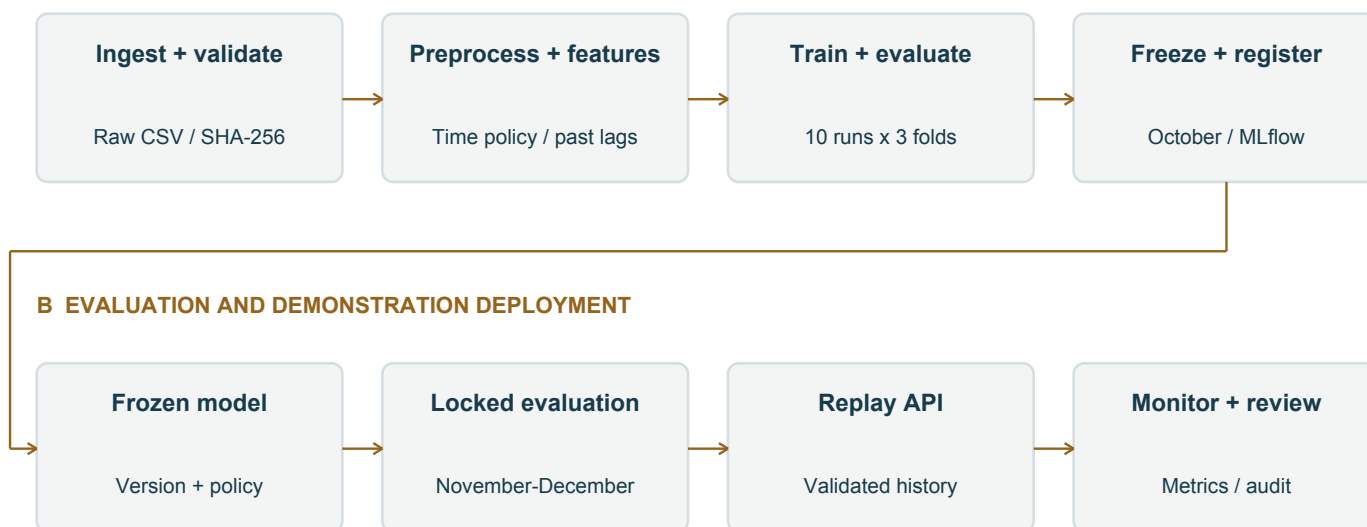
The classroom stack uses a single-worker MLflow server with SQLite and named Docker volumes, rather than PostgreSQL and external object storage. At this dataset size the simpler stack is easier for an assessor to reproduce. Airflow and application dependencies are isolated to avoid conflicting SQLAlchemy/Flask requirements. Human release approval is represented by a candidate-only registry policy, with no automatic champion alias.

All reported model scores come from actual runs. The prospective 5% peak-demand reduction and financial benefits in Assignment 1 remain untested business targets.

1 PIPELINE DESIGN | 25%

A modular pipeline with visible gates

A DEVELOPMENT AND MODEL FREEZE



Artifacts and compact manifests cross stages; no DataFrames are passed through Airflow XCom.

Figure 1. Implemented historical pipeline. Each stage persists a compact manifest and evidence artifacts. Registration precedes demonstration deployment, not operational approval.

Architecture and reuse

The CLI is the common execution boundary for Docker and Airflow. `data.py` owns validation and temporal features; `models.py` owns the predeclared estimators and non-negative prediction rule; `metrics.py` owns error definitions and gates; `experiments.py` owns development, calibration and test freezing. `serving.py` reuses the same feature and prediction functions. This prevents training and inference from quietly using different lags or clipping rules.

Persistence, recovery and scale

Raw data remains unchanged. Derived features are stored as Parquet; predictions and experiment summaries as CSV; stage manifests as JSON. Atomic manifest replacement marks successful completion, while a file lock serialises writers on the shared local volume. Completed stages are reused only when source, configuration and data hashes match. A failed development attempt can leave traceable MLflow runs; the completed manifest identifies the runs used by the report.

Single-machine CPU execution is sufficient for 34,368 feature-ready rows. At larger scale, partition immutable snapshots by plant/time, fan out independent development configurations and use distributed locks plus PostgreSQL/object storage. The single-worker SQLite demo is not presented as a multi-host or highly available design.

1 PIPELINE DESIGN | 25%

Stage inputs, outputs and quality gates

Stage / module	Input -> output and operations	Quality gate / recovery
Ingestion experiments.prepare	Original UCI CSV -> identified source snapshot. Compute SHA-256 and load required fields.	Reject any byte-level difference from the declared dataset; retain the original for attribution.
Validation data.load_meter	Rows -> validated energy series. Parse date, numeric kWh and timestamp keys.	Reject empty, negative/non-finite energy, duplicates or a non-uniform 15-minute grid.
Preprocessing data.load_meter	Raw date -> assumed interval end, with the original CSV retained. Literal-time policy is a separate study.	Never silently sort the primary stream; require the declared normalisation to produce ordered intervals.
Features data.build_features	Validated series -> calendar encodings, energy lags and shifted rolling means.	Whitelist predictors, exclude target-time proxies, drop common weekly warm-up and purge unavailable training labels.
Training experiments.develop	Three chronological training/validation pairs -> ten configurations and fold predictions.	Fit transforms on training only. Use equal evaluation timestamps, seed 42 and a single CPU thread.
Evaluation calibrate / final	Development winner -> October policy freeze -> locked November-December evidence.	Require explicit final confirmation; verify frozen model checksum; no selection on test outcomes.
Registration + serving tracking / serving	Frozen sklearn model -> MLflow version, local bundle and replay API output.	Registry prediction parity, valid historical request, model SHA-256 and candidate-only release status.

Error handling is explicit

Schema and information-boundary failures stop the stage. Missing source data is not imputed from the future or replaced with zero. Airflow retries a failed command once after 15 seconds; repeated deterministic errors remain visible in its task logs. A changed study identity requires a new output directory. Final evaluation can retry the same frozen computation after an interruption, but may not change the selected parameters or advisory threshold.

The replay API returns an error for invalid input or an unavailable bundle. A live seasonal fallback and deadline-aware meter adapter were proposed in Assignment 1 but are not falsely represented as deployed in this historical implementation. Existing plant controls remain outside the project.

1 PIPELINE DESIGN | 25%

Temporal integrity and feature engineering

Data facts and the midnight ambiguity

The licensed UCI dataset contains 35,040 rows and 11 columns, with no empty cells or duplicate literal timestamps [1, 2]. Each source day ends with a 00:00 row carrying that same date. The primary convention advances that trailing midnight by one day and treats records as interval-end totals. It produces a complete grid for 2018 interval starts. This is a documented interpretation, not independently confirmed meter documentation.

The sensitivity study instead sorts the literal timestamps and reruns all ten development configurations. It does not open an alternative test set or choose whichever timestamp convention gives the nicest test score. Both the transformation and its limits are recorded in the manifests.

Feature family	Specification	Availability
Calendar	Sine/cosine of 96 daily slots and seven weekdays; weekend indicator.	Known before the target interval; regenerated after normalisation.
Energy history	Lags 2, 3, 4, 5, 6, 9, 17, 96 and 672.	Target j may read only j-2 or earlier.
Rolling summaries	Means of 4, 16 and 96 readings, ending at j-2.	Shift first, then roll; no centred or future-filled windows.
Excluded columns	CO2, reactive energy, power factors and Load_Type.	Same-target values are unavailable at issue time or may encode the answer.

```
for lag in LAGS:
    out[f"lag_{lag}"] = out.energy.shift(lag)
for window in WINDOWS:
    out[f"mean_{window}"] = (
        out.energy.shift(2).rolling(window).mean()
    )
return out.dropna(subset=feature_columns()).copy()
```

Implementation excerpt: steel_energy/data.py. The test suite perturbs all values from j-1 onward and verifies that row j's predictor vector is unchanged.

Dropping the initial 672-interval warm-up leaves 34,368 feature-ready rows under the primary convention. All configurations, including ablation and baselines, use that same eligible set. Training labels are additionally purged when their interval end plus the assumed 30-second arrival delay exceeds the first validation issue time.

2 EXPERIMENT TRACKING AND METRICS | 25%

A predeclared ten-configuration study

Run	Algorithm / intervention	Configuration
E01	Persistence	Latest completed energy: target lag 2.
E02	Daily seasonal	Target lag 96.
E03	Weekly seasonal	Target lag 672.
E04	Ridge	alpha=1.0; training-only StandardScaler.
E05	Random forest	200 trees; depth=8; minimum leaf=5.
E06	Random forest	200 trees; unrestricted depth; minimum leaf=2.
E07	Histogram boosting	300 iterations; rate=0.05; 15 leaves; L2=1.
E08	Histogram boosting	E07 with 31 leaves.
E09	Histogram boosting	E08 with learning rate=0.10.
E10	Feature ablation	E08 with calendar + short lags only; remove rolling and daily/weekly lags.

E04-E09 use the full 17-feature set: five calendar variables, nine lags and three rolling means. E10 keeps 12 calendar/short-lag features. Baselines remain independent forecasting rules. All regressors use the same non-negative clipping rule. Random state is 42 where supported; histogram boosting disables random internal early-stopping validation. Numerical libraries and tree workers are limited to one thread.

Splits and the study lock

Window	Role	Observed size
July / August / September	Expanding-window validation; fit only earlier available labels.	8,832 validation intervals pooled across three folds.
Through September	Refit the development winner, before October begins.	25,535 eligible training labels after boundary purging.
October	Calibrate margin using the fixed model; do not reselect its algorithm.	2,976 intervals.
November-December	Locked evaluation of the frozen model and policy.	5,856 intervals under the primary convention.

No random split is used. The primary selection criterion is pooled validation MAE, with experiment ID as a deterministic exact-tie breaker. The strongest baseline is selected by the same development criterion. Earlier observations from the held-out period can feed later lagged predictions only after their assumed arrival time; model weights and policy parameters remain fixed.

2 EXPERIMENT TRACKING AND METRICS | 25%

Measured development results and selection

Run	MAE kWh	RMSE kWh	WAPE %	Bias kWh	Fit + predicts
E01	7.802	17.638	33.11	+0.000	0.01
E02	13.538	26.035	57.45	+0.135	0.01
E03	12.962	24.920	55.01	+0.805	0.01
E04	8.571	14.437	36.37	+0.072	0.02
E05	5.032	10.666	21.35	+0.541	32.25
E06	4.901	10.487	20.80	+0.406	64.85
E07	5.089	10.399	21.59	+0.379	0.95
E08	4.960	10.331	21.05	+0.232	1.28
E09	5.039	10.445	21.39	+0.182	1.21
E10	5.025	10.538	21.32	+0.561	1.05

Table 1. All 10 configurations evaluated on the same 8,832 out-of-time rows. Times are the sum of fit and batch-predict calls across three folds, excluding MLflow/network overhead.

What the results support

E01 is the strongest baseline. Repeating yesterday or last week's demand performs substantially worse, consistent with operational activity changing across days. Ridge reduces large misses relative to persistence but has higher MAE, illustrating that improvement in one error criterion does not imply improvement in all criteria.

E06 has the lowest pooled MAE and is selected according to the predeclared rule. E08 achieves lower RMSE than the selected random forest, but its MAE is 1.21% higher. On this measured runtime, E06 takes about 50.5 times E08's fit/predict time. E08 is therefore a credible lower-cost alternative for a later study; the current experiment does not switch to it after inspecting test results.

E10's removal of rolling and seasonal-history features changes MAE from 4.960 to 5.025 kWh. This is evidence about this particular feature combination, not a causal ranking of each individual feature. A separate ablation study would be needed to isolate their effects.

Metric strategy and business alignment

```
MAE = mean(abs(prediction - actual))
RMSE = sqrt(mean((prediction - actual)^2))
WAPE (%) = 100 * sum(abs(prediction - actual)) / sum(actual)
Bias = mean(prediction - actual)
```

MAE is the primary metric because kWh per interval is interpretable to an energy manager. RMSE highlights large misses; WAPE summarises relative error; signed bias detects systematic over- or underprediction. High-load error and alert precision/recall address the actual advisory use case. MAPE is avoided because zero and near-zero energy occurs [3]. Neither forecasting error nor alert recall demonstrates reduced electricity bills.

2 EXPERIMENT TRACKING AND METRICS | 25%

Result patterns and timestamp sensitivity

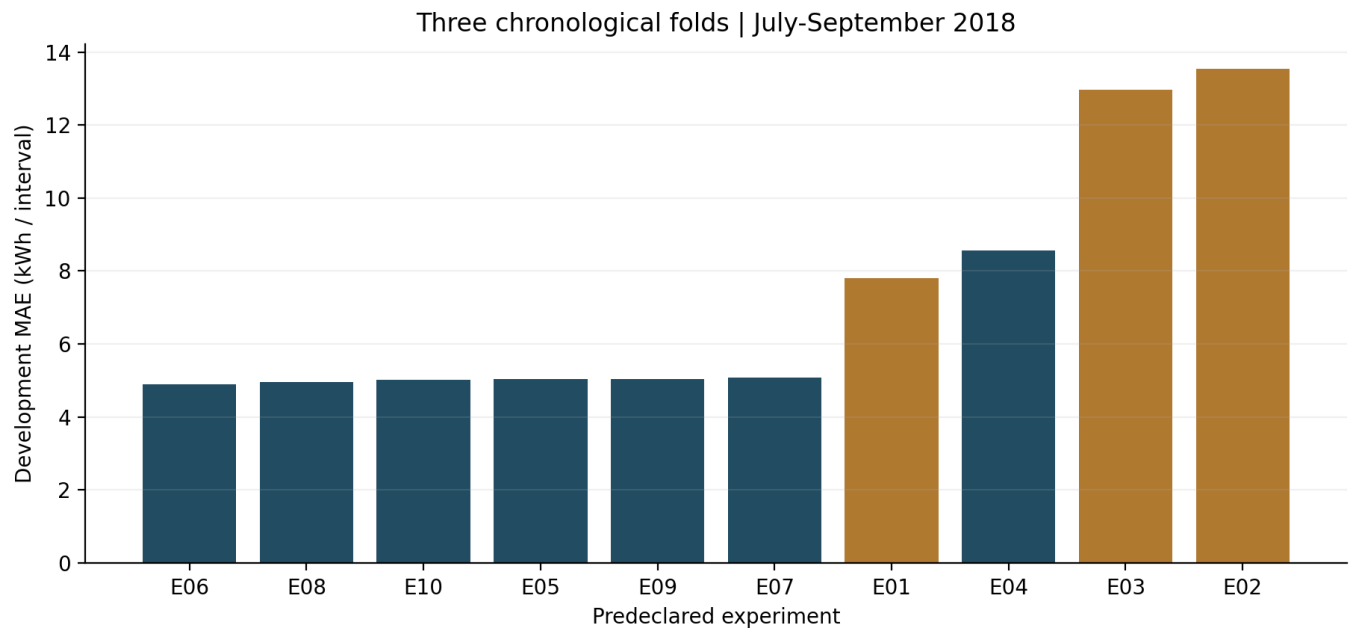


Figure 2. Primary development MAE. Amber bars are deterministic baselines; blue bars are learned models. Lower is better.

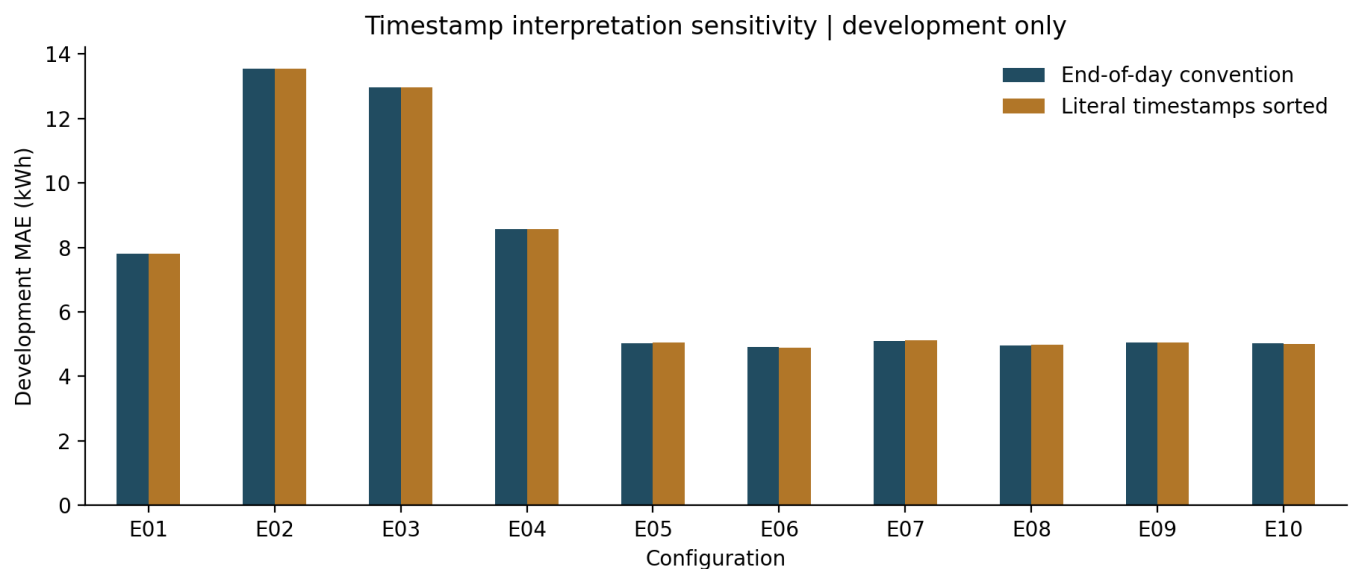


Figure 3. Both timestamp conventions are evaluated using development data only.

The literal-time sensitivity study selects E06; the primary study selects E06. The largest absolute configuration-level MAE change is 0.020 kWh. The selected model's MAE changes from 4.901 to 4.893 kWh. This supports ranking stability for this comparison; it does not resolve the source meter's true midnight convention.

2 EXPERIMENT TRACKING AND METRICS | 25%

Locked-test evidence and its uncertainty

Metric	Frozen E06	Baseline E01
MAE (kWh)	4.703	7.515
RMSE (kWh)	9.766	16.931
WAPE (%)	18.908	30.214
Signed bias (kWh)	0.030	0.000
High-load MAE (kWh)	19.287	24.849

The frozen model reduces test MAE by 37.42% relative to the previously chosen baseline. There are 5,856 observations across 61 target dates. Paired daily MAE reduction is 2.812 kWh; its empirical 95% moving-block bootstrap interval is [2.201, 3.535] kWh, using 2,000 resamples, seven-day blocks and seed 42.

The paired block analysis preserves local within-week dependence better than treating every 15-minute row as independent. It is an exploratory uncertainty estimate on one historical plant/year, not a guarantee about a different plant or future production regime.

Slice	n	Model MAE	Baseline MAE	Model WAPE %
November	2880	5.713	9.417	19.09
December	2976	3.725	5.675	18.65
weekday	4128	5.547	9.331	18.05
weekend	1728	2.688	3.176	24.68
high_load	361	19.287	24.849	18.82

November has higher absolute error than December, while weekend MAE is lower but weekend WAPE is higher because the energy denominator is smaller. High-load error remains much larger than overall MAE. These slices explain why aggregate forecast improvement alone is insufficient to approve a high-load advisory.

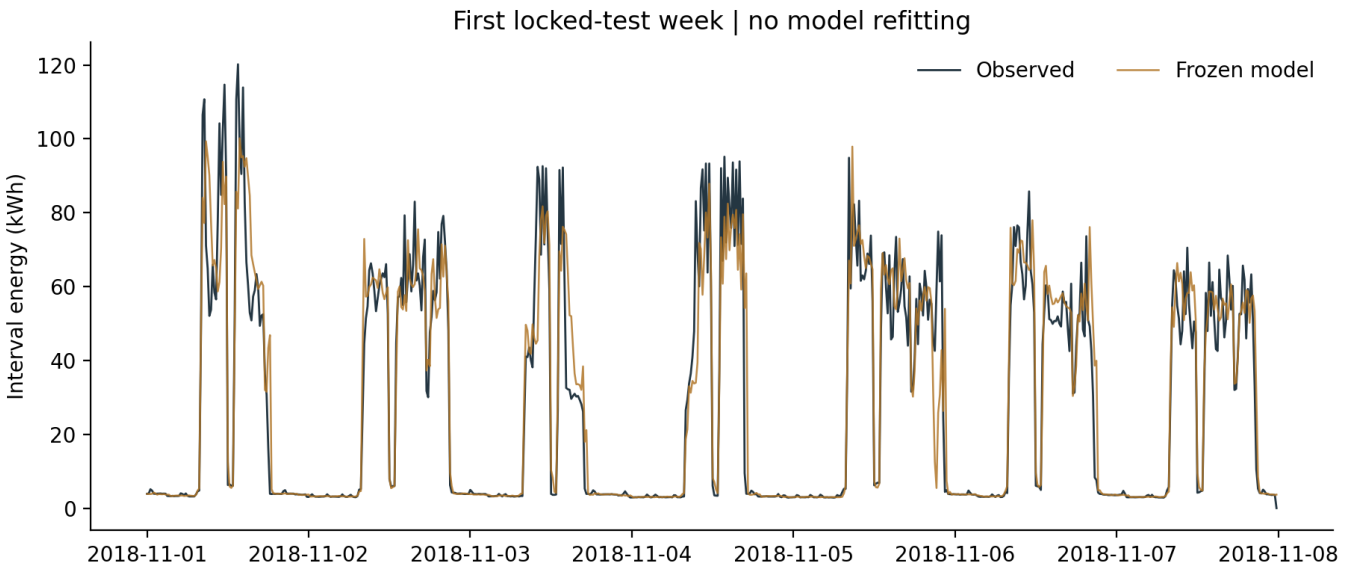


Figure 4. The first locked-test week. Abrupt load changes remain difficult; the model is not refitted during the test.

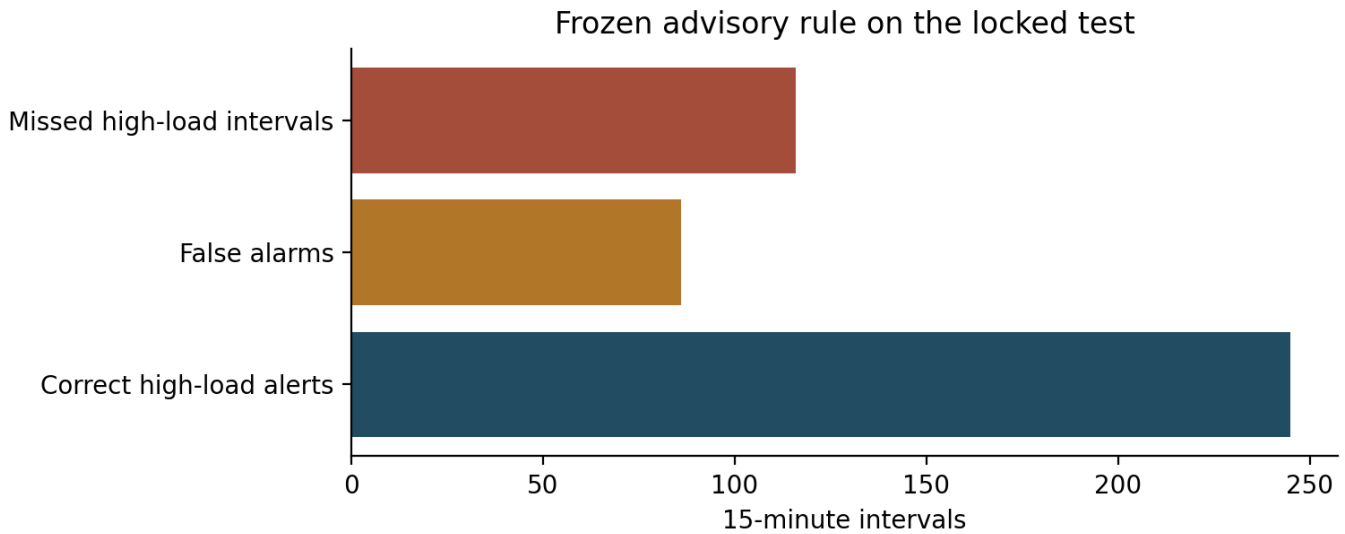
2 EXPERIMENT TRACKING AND METRICS | 25%

Advisory calibration and release decision

The high-load threshold is the 90th percentile of the final development training labels: 80.484 kWh per interval. This is an analytical definition, not an electrical limit or tariff demand threshold. The fixed model is evaluated on October using the predeclared margin grid; the smallest margin meeting recall $\geq 80\%$ and precision $\geq 60\%$ is chosen.

Margin kWh	October recall	October precision	Alert count
0	60.0%	69.0%	239
2	64.7%	67.7%	263
4	70.5%	66.9%	290
6	71.6%	64.8%	304
8	73.1%	62.8%	320
10	75.3%	59.8%	346

The frozen margin is 0 kWh. Calibration feasibility is false. If no margin qualifies, the implementation records failure and retains zero only for diagnostic scoring; it does not enable a production advisory. Locked-test precision is 74.0% and recall is 67.9%, with 361 high-load intervals.



Honest promotion gate

The forecast gate passes: it jointly requires $MAE \leq 5$ kWh, at least 10% relative MAE improvement and high-load MAE no worse than the baseline. The advisory gate does not pass: October must be feasible and locked-test recall/precision must meet their thresholds. The registered model remains a candidate, and the replay API always reports `production_approved=false`. No threshold is relaxed after seeing these results.

A live rollout would additionally require confirmed meter timing, a local shadow period, operator-response evidence and production/tariff records. Lower historical error cannot establish the proposed peak-demand reduction or savings. The report recommends the measured development winner for further validation, subject to these gates and limitations.

2 EXPERIMENT TRACKING AND METRICS | 25%

MLflow tracking, artifacts and model registry

Each timestamp policy has a separate experiment. Each configuration has a parent run and three nested fold runs; the primary study additionally records a frozen-candidate run and a locked-test evaluation run. Parameters, fold boundaries, row counts, pooled errors, fit/predict time and prediction artifacts are retained. Data, configuration and source hashes are tags; JSON manifests retain undefined metrics as null rather than inventing a numeric value.

```
mlflow.set_tracking_uri(uri)
mlflow.set_experiment(name)
with mlflow.start_run(run_name=f"{spec['id']}-development"):
    mlflow.log_params(**spec, "seed": config["seed"])
    mlflow.set_tags(**identity(), "stage": "development")
    mlflow.log_dict(environment(), "environment.json")
    # Each temporal fold is a nested run.
    with mlflow.start_run(run_name=start[:7], nested=True):
        log_metrics(metrics)
    mlflow.log_artifact(str(prediction_csv))
```

Condensed from tracking.py and experiments.py. prediction_csv denotes the configuration's saved development-prediction file; the full executable paths are in the project.

```
mlflow.sklearn.log_model(
    model, artifact_path="model",
    signature=infer_signature(train[cols], model.predict(train[cols])),
    input_example=train[cols].head(2),
    code_paths=[str(ROOT / "steel_energy")],
    pip_requirements=recorded_requirements,
)
version = mlflow.register_model(
    f"runs:{run.info.run_id}/model", config["registered_model"]
)
client.set_registered_model_alias(
    config["registered_model"], "candidate", version.version
)
```

Registration excerpt, with recorded_requirements abbreviated for readability. It is generated from measured dependency versions in the implementation.

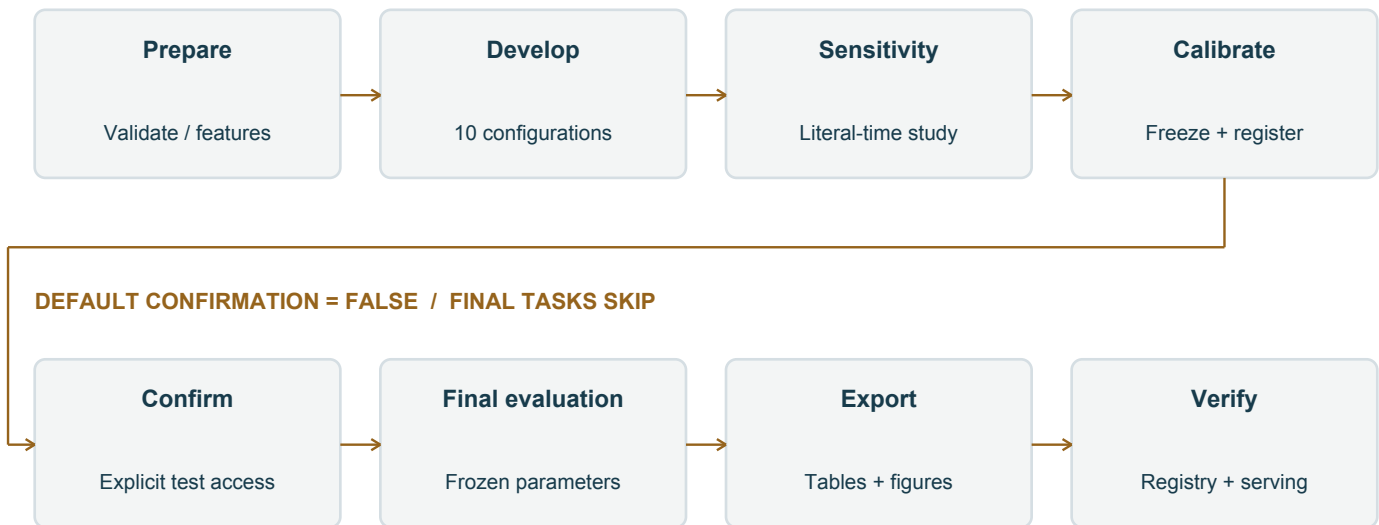
This report references model steel-energy-interval-forecast version 1, run 1c218d8163c748f9b588edbbd622a1a0. A fresh assessor run creates its own version. Verification reloads the explicit registered version, compares 20 predictions against the local bundle and checks the model checksum. Serving applies the same non-negative clipping used in evaluation.

The Compose MLflow server proxies artifact uploads/downloads over its internal HTTP service. The client does not require access to a server-local filesystem path. Metadata and artifacts persist in a named volume; this avoids machine-specific absolute paths in the lecturer workflow.

3 WORKFLOW ORCHESTRATION DESIGN | 20%

Airflow DAG and scheduling strategy

RESEARCH DAG / MANUAL TRIGGER



Artifacts and compact manifests cross stages; no DataFrames are passed through Airflow XCom.

Figure 5. Eight-task research DAG. Explicit confirmation is required before the locked-test task; downstream tasks skip when confirmation is false.

Task	Responsibility / dependency
ingest_validate_features	Check raw checksum, temporal grid and feature artifact; produces prepared.json and Parquet.
ten_configuration_development	Run or verify the complete primary development matrix after valid features exist.
timestamp_sensitivity	Run the separate literal-time development study; no alternate test selection.
freeze_model_and_advisory	Refit the selected candidate, calibrate October, register and freeze the bundle.
confirm_locked_test	Require confirm_final=true; otherwise raise AirflowSkipException.
locked_test_evaluation	Score the fixed model/policy with explicit final confirmation.
export_figures	Generate charts only from saved measured outputs.
verify_registry_and_serving	Check registered-model parity and create the replay request/output.

The historical research DAG has schedule=None, catchup=False and max_active_runs=1. Automatic weekly reruns of the same historical holdout would not create fresh evidence, so training is deliberately manual. A second paused-by-default DAG verifies frozen evidence daily at 06:00 UTC when enabled. The Assignment 1 live design's 15-minute inference and weekly training schedules remain future work requiring new meter data.

3 WORKFLOW ORCHESTRATION DESIGN | 20%

Executable tasks, recovery and monitoring

```
with DAG(
    "steel_energy_research",
    start_date=datetime(2026, 1, 1, tzinfo=timezone.utc),
    schedule=None, catchup=False, max_active_runs=1,
    params={"confirm_final": Param(False, type="boolean")},
    default_args={
        "retries": 1, "retry_delay": timedelta(seconds=15),
        "execution_timeout": timedelta(minutes=30),
        "on_failure_callback": log_failure,
    },
) as research_dag:
    def stage(task_id: str, arguments: str) -> BashOperator:
        return BashOperator(
            task_id=task_id,
            bash_command=(
                f'cd "{ROOT}" && "{PYTHON}" '
                f'-m steel_energy.cli {arguments}'
            ),
        )
```

```
prepare >> develop >> sensitivity >> calibrate
calibrate >> approval >> final >> plots >> verify

def require_final_confirmation(**context) -> None:
    if not context["params"]["confirm_final"]:
        raise AirflowSkipException("Final test remains locked")
```

Condensed executable definitions from dags/steel_energy_training.py. Runtime paths refer to the container, not to the author's laptop.

Failure handling and observability

Every failed task has one retry after 15 seconds and a 30-minute execution timeout. Commands fail with non-zero status on invalid input, stale identities or incompatible artifacts. Structured callback logs include DAG, task, run and log URL. Airflow exposes task states and duration; MLflow exposes run metrics and artifacts. External email/pager integration is a production extension, not an enabled notification service in this submission.

The project uses SequentialExecutor for a small, single-host demonstration. Application work runs under the separate core Python runtime while Airflow uses its own constrained environment. This avoids library conflicts and keeps the command path identical to CLI execution. Hash-checked stage reuse makes retrying a completed command safe; the file lock prevents simultaneous writers from corrupting the shared study.

4 CODE QUALITY AND DOCUMENTATION | 20%

Clear contracts, configuration and tests

Modules are separated by responsibility, core data and pipeline operations use docstrings and type hints, and comments explain only non-obvious constraints such as the two-interval information gap and timestamp assumption. Ruff checks imports and common Python mistakes; formatting is applied consistently. README documents the lecturer's Docker commands, outputs, ports, stop procedure, known limitations and recovery instructions.

```
def temporal_split(
    frame: pd.DataFrame, start: str, end: str
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Purge labels unavailable at the first issue time."""
    valid = frame.loc[
        frame.interval_start.ge(start)
        & frame.interval_start.lt(end)
    ].copy()
    train = frame.loc[
        frame.interval_start.lt(start)
        & (frame.interval_end + pd.Timedelta(seconds=30))
        .le(valid.issue_time.min())
    ].copy()
    return train, valid
```

Excerpt from data.py; the complete implementation also rejects empty windows.

```
seed: 42
experiment_name: steel-energy-assignment2
registered_model: steel-energy-interval-forecast
calibration: ["2018-10-01", "2018-11-01"]
test: ["2018-11-01", "2019-01-01"]
alert_margins: [0, 2, 4, 6, 8, 10]
```

```
path = Path(os.getenv(
    "CONFIG_PATH", ROOT / "config/experiment.yaml"
))
config = yaml.safe_load(path.read_text())
output = Path(os.getenv("OUTPUT_DIR", ROOT / "artifacts/run"))
uri = os.getenv("MLFLOW_TRACKING_URI", local_sqlite_uri)
```

The config file fixes all ten configurations and evaluation dates; environment variables override locations and service addresses. Optional .env values change host ports only. No host-specific paths or secrets are required. Behavioural tests cover missing/duplicate/negative data, the midnight boundary, lag arithmetic, future perturbations, fitting cutoffs, undefined metric denominators, release guards and serving rejection of late/future inputs.

5 REPRODUCIBILITY AND VERSIONING | 10%

Source, data, model and container lineage

Source is submitted through the public GitHub repository below; this PDF is the written deliverable. Short-lived feature branches, reviewed main and release tags form the proposed Git workflow. CI builds Docker and runs tests/lint. Reference experiments predate Git publication and retain their original source/configuration hashes; no retrospective commit or hosted CI result is invented.

<https://github.com/hiepdqflame/ddm501-assignment2-steel-energy>

Record the submitted commit SHA or release tag with this URL. A later software revision does not make the historical test set unseen again.

```
git switch -c feature/energy-pipeline
git add steel_energy tests config dags Dockerfile compose.yaml
git commit -m "Implement reproducible energy pipeline"
git tag -a v1.0.0 -m "Assignment 2 submission"
git rev-parse HEAD
```

Proposed release commands. See docs/versioning.md for the full branch, review and rollback strategy.

The original CSV checksum is pinned; derived Parquet and fitted model have separate checksums. Immutable MLflow versions link the trained estimator to parameters, dependency versions and frozen alert policy. The candidate alias aids inspection, while replay verifies an explicit version and local checksum. A future rollback must restore model, schema and policy together.

```
# Dockerfile - selected application lines
FROM python:3.12-slim-bookworm@sha256:<pinned-index> AS app
ENV OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
WORKDIR /app
COPY requirements.lock ./requirements.lock
RUN pip install --no-cache-dir -r requirements.lock
RUN pip install --no-cache-dir greenlet==3.5.6
COPY steel_energy ./steel_energy
COPY config ./config
COPY data ./data
CMD ["python", "-m", "steel_energy.cli", "all",
    "--confirm-final", "--sensitivity"]
```

The full Dockerfile contains the actual multi-platform digest, additional test/report files and the isolated Airflow stage. The long digest is abbreviated here only for legibility.

```
# Explicit randomness and execution limits in the implementation
model = RandomForestRegressor(
    **params, random_state=seed, n_jobs=1
)
with threadpool_limits(limits=1):
    model.fit(train[cols], train.energy)
rng = np.random.default_rng(config["seed"])
```

Measured application versions include Python 3.12.14, scikit-learn 1.7.2, NumPy 2.2.6, pandas 2.2.3 and MLflow 2.22.2. Airflow is pinned to 2.10.5 with its official Python 3.12 constraints. Fixed seeds and one thread reduce variation but do not promise identical timings or bitwise equality across BLAS/CPU architectures.

Data SHA-256: 9b1cee6f9cb9cd9df2b95814ca90a9a2ff15b7f5f1fba0fae3c643e82072eacc
Source/config/model checksums and runtime details are retained in frozen.json and MLflow artifacts.

VERIFICATION AND ASSESSOR WORKFLOW

Reproduce the study and inspect the evidence

```
# From the submitted project folder
docker compose up -d --build mlflow
docker compose run --build --rm pipeline
docker compose run --rm --no-deps pipeline \
  python -m pytest tests -q
docker compose --profile demo up -d --wait api
docker compose run --rm --no-deps pipeline \
  python scripts/check_api.py
```

The project bundles the original licensed CSV. The initial build needs network access for images and dependencies; subsequent execution uses the bundled data and internal services. MLflow is exposed at localhost:15030, the replay API at localhost:18030 and optional Airflow at localhost:18081. Services bind to loopback. Host ports are configurable without changing container addresses. A clean ZIP extraction with an independent registry reproduced all 5,856 final predictions exactly on the same machine.

Initial verification on 25 September 2026: 17 contract tests passed in Linux ARM64 and emulated AMD64 containers. The full measured study runs on ARM64. The research DAG succeeds with explicit confirmation; without it, final evaluation and downstream tasks skip. The monitor DAG succeeds. The engineering upgrade adds integrity/cache/recovery tests and HTTP integration checks; the supplemental verification page distinguishes these newer checks from the original cross-platform run.

Evidence	What it proves	What it does not prove
Unit/contract tests	Time boundaries, future-data exclusion, malformed-input rejection and metric edge cases.	All possible production data or failures are covered.
Docker pipeline run	Declared code/data/dependencies can execute the measured study in a Linux container.	Identical runtime on every lecturer machine or a production SLA.
MLflow round trip	The explicit registered model matches 20 local predictions after loading.	A moving alias is automatically safe to promote.
Airflow DAG integration	Task definitions, dependencies, confirmation and CLI boundaries execute against persisted evidence.	Distributed scheduling or live meter arrival deadlines are validated.
HTTP replay check	An actual API response matches the saved local replay prediction.	The API is connected to a real plant or authorised to control equipment.

Warm single-row model-call p95 is 2.410 ms on the recorded runtime. This measurement excludes data ingestion, scheduling, network and HTTP overhead and therefore cannot establish the Assignment 1 end-to-end deadline or API service target. The runtime and exact verification commands are preserved with the project.

Generated outputs are separate from reference-results. A fresh assessor run trains models and creates its own registry records. A repeated run can safely reuse matching stage evidence; changed identities are rejected. To perform a separate software reproduction, use a new OUTPUT_DIR instead of replacing the original study. The same historical holdout does not become scientifically unseen again.

CONCLUSIONS, LIMITATIONS AND REFERENCES

A reproducible result with a bounded claim

The completed study recommends Eo6 under the declared MAE objective. It achieves 4.703 kWh locked-test MAE versus 7.515 for Eo1; the forecast gate passes and the advisory gate does not pass. The recommendation is a candidate for further validation, with the complete result and any failed conditions retained. Eo8 remains a useful cost/accuracy alternative for a separately designed future experiment.

Limitations are material: one plant, one historical year, an unconfirmed interval-end convention, assumed arrival timing and no production/tariff/action records. Timestamp sensitivity checks model-ranking stability but cannot resolve the original meter contract. High-load events are an analytical percentile definition. Operator response, real peak reduction, financial benefit, live fallback and high availability require additional prospective evidence.

[1] Sathishkumar V E, Shin, C., and Cho, Y. Steel Industry Energy Consumption. UCI Machine Learning Repository. DOI: 10.24432/C52G8C. Accessed 24 September 2026.
<https://archive.ics.uci.edu/dataset/851/steel+industry+energy+consumption>

[2] Creative Commons. Attribution 4.0 International (CC BY 4.0). Dataset licence stated on the UCI page. Original CSV retained; derived features and time conventions are identified as project transformations.
<https://creativecommons.org/licenses/by/4.0/>

[3] Hyndman, R. J., and Athanasopoulos, G. Forecasting: Principles and Practice, 3rd edition. Forecast accuracy and time-series cross-validation. <https://otexts.com/fpp3/accuracy.html>

[4] MLflow documentation. Tracking and Model Registry; implementation uses MLflow 2.22.2.
<https://mlflow.org/docs/latest/ml/model-registry/>

[5] Apache Airflow documentation. DAGs, task dependencies and best practices; implementation uses Airflow 2.10.5.
<https://airflow.apache.org/docs/apache-airflow/2.10.5/best-practices.html>

[6] Scikit-learn documentation. Common pitfalls and recommended practices, including training-only preprocessing and data leakage. https://scikit-learn.org/stable/common_pitfalls.html

Traceability to the assessment

Criterion	Evidence in this report / project
Pipeline design - 25%	Figure 1, stage specifications, feature/time contracts and modular implementation.
Experiments and metrics - 25%	Ten-run matrix, measured tables/plots, MLflow snippets, frozen test and recommendation.
Workflow orchestration - 20%	Figure 5, executable DAG, trigger policy, failure handling and monitoring.
Code quality - 20%	Typed/documented modules, YAML/env loading, tests, lint and Docker README.
Reproducibility - 10%	Git strategy, checksums, pinned environment, seeds, registry versions and Dockerfile.

SUPPLEMENT A | CRITICAL ERROR ANALYSIS

Why the alert remains a research candidate

This analysis was added after the original model and test predictions were frozen. Test errors are described, not used to select new features, parameters or alert thresholds. All original study artifacts retain their checksums. The separate October analysis probes the existing model without refitting it.

Missed-peak diagnostic	Measured value
False-negative intervals / high-load intervals	116 / 361
Consecutive missed episodes / longest episode	81 / 3 intervals
Misses above the last available reading	108 / 116
Median rise from last available reading	25.47 kWh
Mean signed error among missed peaks	-25.48 kWh (forecast minus actual)

Most misses coincide with rising load relative to the j-2 reading available at issue time. This is consistent with delayed information during rapid changes; it does not identify a causal mechanism. A future prospective study could evaluate genuinely available production schedules or ramp-aware models using new validation and test periods. The current alert gate remains unchanged.

Grouped permutation sensitivity on October only

Feature group	MAE increase, kWh	Repeat SD
Recent energy (30-90 min)	17.874	0.615
Time of day	3.977	0.212
Previous week	1.242	0.102
Rolling means	0.874	0.097
Previous day	0.483	0.107
Earlier energy (135-255 min)	0.196	0.013
Day of week	0.090	0.018

The frozen model's unperturbed October MAE is 6.229 kWh. Groups are permuted jointly in 16-row (four-hour) blocks, five times with seed 42. Within-block order is preserved. Recent energy has the largest observed impact, followed by time of day; this helps explain sensitivity to abrupt changes beyond observed history.

Correlated groups and unrealistic cross-group combinations limit interpretation. Repetition SD reflects permutation variation, not a confidence interval or causal feature effect. No permutation is computed on the final test set.

SUPPLEMENT B | ENGINEERING VERIFICATION

Serving performance and explicit failure behavior

Verified cache and immutable completed stages

The API holds the approximately 192 MiB estimator in memory. A lock serializes reloads; file metadata and the small manifest's actual bytes identify a revision, including rapid writes with identical timestamps. Publish model replacements atomically; never edit an active model in place. A changed bundle triggers checksum verification and deserialization of the same open file; an update during loading is rejected. Corrupt or missing files return 503 instead of the previous cached model. Histories are bounded to exactly 672 readings; input-contract failures return 422.

A separate integrity ledger records the dependency lockfile, actual installed versions, Python, operating system and CPU architecture, plus SHA-256 hashes for artifacts belonging to completed stages. Changed completed evidence or an incompatible runtime fails reuse. Incomplete stages remain retryable. The original study was adopted only through an explicit consistency audit; historical manifests and the six-module experiment identity were not rewritten. Local checksums do not protect against a malicious actor replacing both files and ledger.

Measured warm HTTP performance

Concurrent clients	Requests	HTTP p50, ms	HTTP p95, ms
1	100	5.75	6.10
4	100	53.90	70.45
8	100	97.98	131.20

Warm HTTP round trip, Compose network, 672 readings/request, one Uvicorn worker. Includes parsing, features, model and transport; excludes meter ingestion. Hardware-specific observation, not an SLA. Ten warm-up requests; 53,407-byte JSON payload; all responses matched the saved forecast.

Parallel requests contend for a single Python process and increase tail latency in this test. Multiple worker processes would each hold their own model copy and increase memory demand; this is a measured trade-off, not an untested scaling claim. HTTP timing still excludes live meter delivery and scheduler delay, so it cannot establish the full plant deadline.

Failure and integration evidence

All 30 expanded tests passed in ARM64 Docker. They cover concurrent cache access, changed versions, missing/corrupt/invalid serialized models, artifact tampering, environment drift and retry after a failed registry-dependent stage. A real HTTP integration fixture exercises 24 requests, 422 input rejection, 503 corruption detection and verified recovery. MLflow tests exercise an unavailable endpoint and successful upload, registration and download from the running service.

The CI workflow additionally waits for Compose service readiness and imports both DAGs in the isolated Airflow runtime, checking eight tasks and false/true confirmation behavior. These commands were exercised locally in Docker. A hosted GitHub Actions run is not claimed. Original AMD64 verification covered the initial 17 tests; the engineering upgrade was verified on ARM64.

SUPPLEMENT C | ASSESSOR DEMONSTRATION

A dashboard grounded in saved evidence

The FastAPI service hosts an offline-capable HTML/CSS/JavaScript interface, with no CDN or separate frontend runtime. It reads the generated study evidence; it does not substitute synthetic data or a preview model for the trained candidate.



Figure 6. Author's Docker dashboard, displaying 19 December 2018. It explicitly separates forecast skill from operational approval.

The assessor can select any of the 61 test days, compare actual/forecast/baseline curves, inspect the ten development configurations and review missed peaks and October feature sensitivity. The historical replay button sends the saved 672-reading request to the running prediction endpoint and reports its response. Availability and loading errors have explicit states.

Chrome browser verification covered 1440-pixel desktop and 390-pixel mobile layouts, date selection and the replay action. No JavaScript errors or page-level horizontal overflow were observed. The chart is horizontally scrollable on narrow screens. API documentation remains available at /docs; source data and model lineage remain visible.